Collaborative Filtering

Dr Seungmin Jin
sedzhin@hse.ru

HSE

Course Objectives

- What is Collaborative Filtering?
  Learn from collective user–item interactions

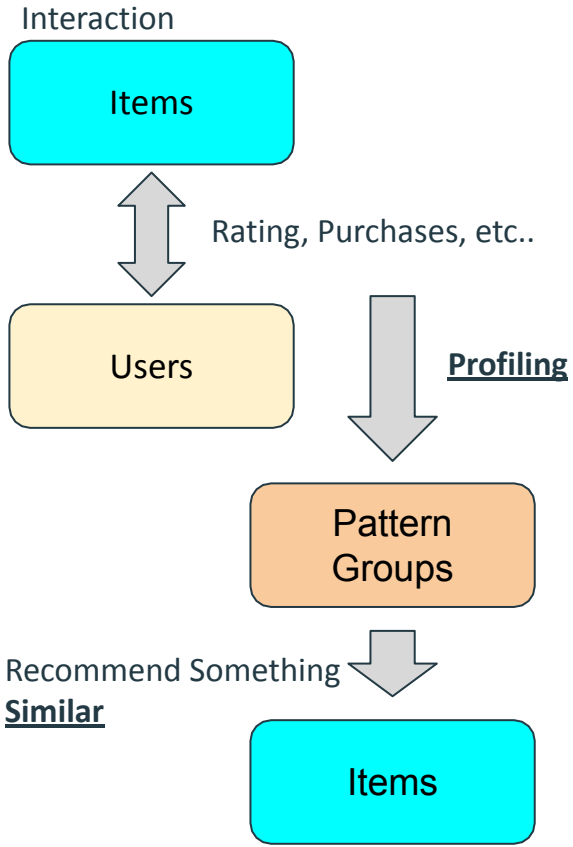
- Two Approaches
  User-based CF: similar users
  Item-based CF: similar items

- Key Limitations
  Cold start · Sparse data

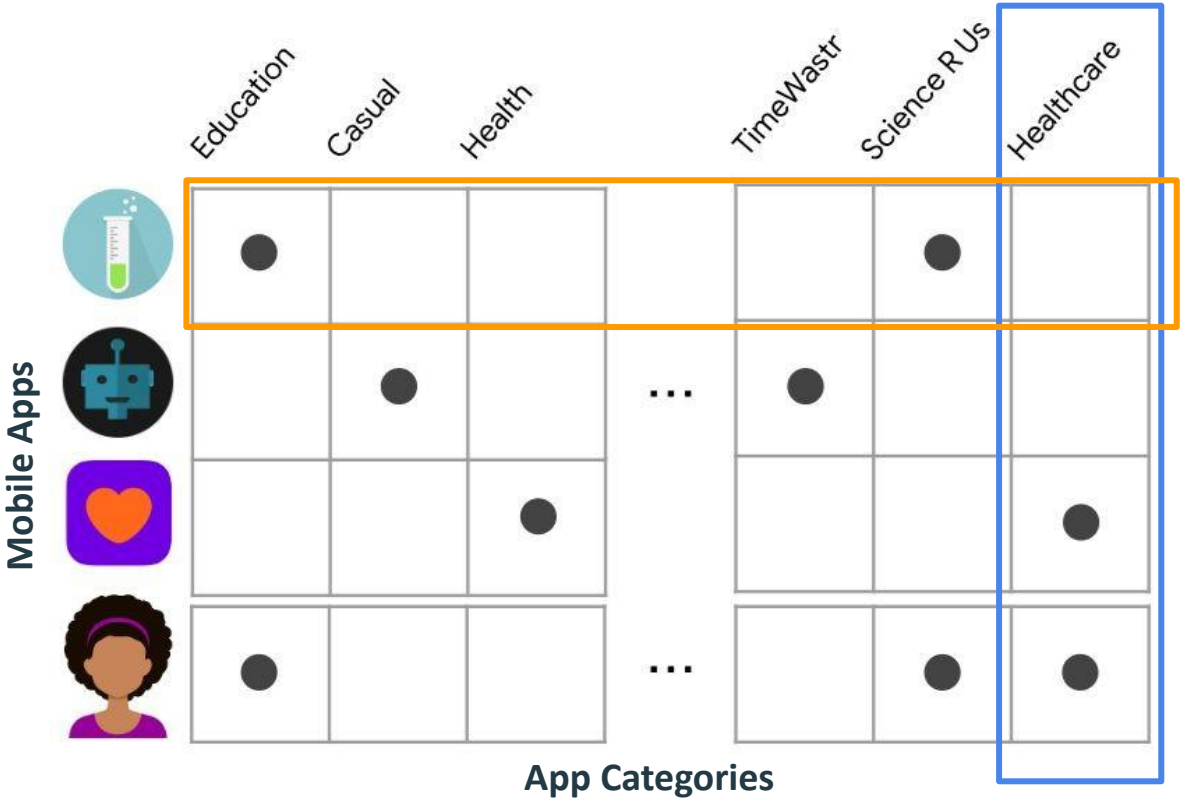
Content-Based Filtering



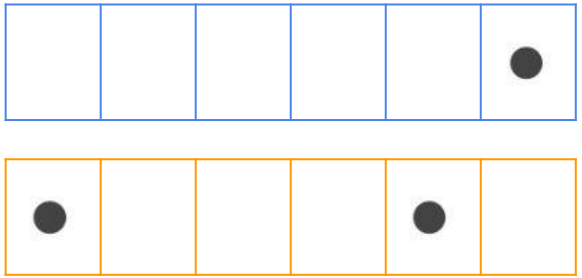
Personal Shopper who knows your tastes!



Content Matrix



Profile Matrix



Average

0.7				0.7	0.3

Where Content-Based Filtering Falls Short

CBF

CBF learns from item attributes. That creates four predictable blind spots.

1 New user: no preference profile



No history → nothing to match against item features.

2 Metadata quality is the bottleneck



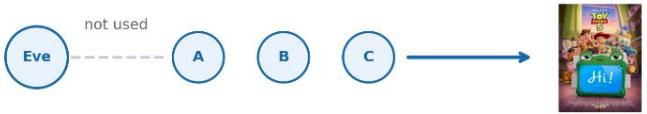
The system can recommend only what its attributes represent.

3 Over-specialization: “more of the same”



Repeatedly recommends nearby items → less novelty and discovery.

4 Cannot exploit collective taste



Misses: “people like Eve also enjoyed this.”

CBF is strongest when rich item features and an established user profile are both available.

Collaborative Filtering







Users

Getting Recommendation from close users!

CHANEL



PRADA
MILANO
DAL 1913

Items

Collaborative Filtering

Today's focus: Memory-Based CF (user-based & item-based).

Techniques	Definition	Advantages	Disadvantages
Memory-Based Approach	Find similar users/items using cosine similarity or Pearson correlation (user-based) / Adjusted Cosine (item-based), then take a weighted average of ratings.	Simple to build. Easy to explain why a recommendation was made.	TODAY'S FOCUS Performance drops when the rating matrix is sparse. Does not scale well to very large catalogs.
Model-Based Approach	Learn latent factors from the rating matrix via Matrix Factorization, PCA/SVD, or neural nets to predict unrated items.	Handles sparse data by compressing it into a small number of latent dimensions.	PREVIEW — NEXT SESSION Latent factors are harder to interpret; training is more computationally expensive.

Cosine → Pearson → Adjusted Cosine

Same idea (remove bias, then compare direction). Each step changes only one thing.

COSINE

Compare raw vectors.

$$\cos(x, y) = \frac{x \cdot y}{\|x\| \, \|y\|}$$

no bias removed

	Toy Story	Star Wars	Dune
Alice	5	4	3
Bob	3	2	1

Question

Do raw rating vectors point in a similar direction?

PEARSON

Compare users. Remove each user's own mean rating.

$$\rho(u, v) = \frac{(r_u - \bar{r}_u)(r_v - \bar{r}_v)}{\|r_u - \bar{r}_u\| \|r_v - \bar{r}_v\|}$$

$\bar{r}_u$ = user\u2019s mean over co-rated items

	Toy Story	Star Wars	Dune
Alice	+1 (5−4)	0 (4−4)	−1 (3−4)
Bob	+1 (3−2)	0 (2−2)	−1 (1−2)

Question

Do users like/dislike the same items relative to their own norm?

ADJUSTED COSINE

Compare items. Remove each rater's global mean rating.

$$sim(i, j) = \frac{(r_{u,i} - \bar{r}_u)(r_{u,j} - \bar{r}_u)}{\|r_{u,i} - \bar{r}_u\| \|r_{u,j} - \bar{r}_u\|}$$

$\bar{r}_u$ = each rater\u2019s global mean

	Bob (mean 4.5)	Carol (mean 2.0)
Dune	+0.5 (5 − 4.5)	+1.0 (3 − 2.0)
Star Wars	−0.5 (4 − 4.5)	0 (2 − 2.0)

Question

Do items get unusually high or low scores from the same raters?

Core idea

Pearson and Adjusted Cosine are both "cosine after centering."

They differ only in what they compare (users vs. items) and whose mean gets subtracted.

User-Based CF: Who Should Influence Alice?

Goal: predict Alice’s rating for Toy Story using people with similar rating patterns.

1. Alice’s known ratings



Star Wars
Alice’s rating
5



Dune
Alice’s rating
3



Interstellar
Alice’s rating
5



Toy Story
Alice’s rating
?

Target: Toy Story = ?

2. Weight each neighbor’s signal

Bob — taste twin
sim +1.00 · Toy Story: 2 · **effect −1.33**

Carol — opposite taste
sim −1.00 · Toy Story: 5 · **effect −2.33**

Dave — mostly aligned
sim +0.87 · Toy Story: 4 · **effect 0.00**

effect = similarity × (neighbor rating − neighbor mean)

3. Combine the evidence

Alice’s baseline
4.33

weighted adjustment
−3.667 / 2.866

=

Predicted rating 3.05

below Alice’s usual rating

Key idea: neighbors matter in proportion to both their similarity and how unusually they rated the target item.

Same History, Different Recommendation Logic

CONTENT-BASED FILTERING

Eve's Watched History



Dune



Interstellar

Averaged Feature Profile

sci-fi	1.0
epic	1.0
space	0.5
desert	0.5



Star Wars

USER-BASED COLLABORATIVE FILTERING

User Ratings

User	Dune	Interstellar	Star Wars
Eve	5	5	–
Bob	5	4	5

Shared pattern:
Both rate Dune and Interstellar highly.



Star Wars

CBF averages item features into a profile.

User-CF compares behavior patterns.

Item-Based CF: Which Items Should Influence Toy Story?

Same ratings, reversed question: compare Toy Story to items Alice has already rated.

1. Alice's known ratings



Star Wars
Alice's rating
5



Dune
Alice's rating
3



Interstellar
Alice's rating
5



Toy Story
Alice's rating
?

Target: Toy Story = ?

2. Weight Alice's rated items

Star Wars opposite pattern
sim −0.99 Alice rating: 5 effect −1.65

Dune moderately co-liked
sim +0.72 Alice rating: 3 effect 0.00

Interstellar opposite pattern
sim −0.84 Alice rating: 5 effect −1.12

effect = item similarity × (Alice rating − item mean)

3. Combine the evidence

Toy Story baseline
3.67

weighted adjustment
−2.764 / 2.543

=

Predicted rating 2.58

below the item's usual rating

Key idea: an item influences the prediction according to its similarity to Toy Story and Alice's deviation from that item's average.

Item-Based CF: Where Does 2.58 Come From?

Toy Story is the target; Alice has rated three other movies.

Rated item	Alice rating	Item mean	Deviation	sim to Toy Story	Effect = sim × Deviation
Star Wars	5	about +1.33*	about +1.67*	−0.99	about −1.65
Dune	3	3.00*	0.00	+0.72	0.00
Interstellar	5	about +3.67*	about +1.33*	−0.84	about −1.12

*Item means inferred from rounded effects for illustration; original slide does not show them.

Glossary

Target:	unrated movie to predict
Baseline:	Toy Story mean = 3.67
Item mean:	an already-rated movie's average across users
Deviation:	Alice rating − that item mean
Similarity (sim):	rating-pattern relationship to Toy Story
Effect:	sim × deviation; one item's unnormalized contribution

$$\text{Predicted rating} = \text{baseline} + \frac{\text{sum(effects)}}{\text{sum(|similarities|)}}$$

$$= 3.67 + (-2.764 / 2.543) \approx 2.58$$

Similarity is computed behorheand, this slide shows prediction, not the similarity calculation.

User-based vs Item-based CF

	User 1	User 2	User 3
Movie 1	5	?	4
Movie 2	?	3	1
Movie 3	4	2	3

	User 1	User 2	User 3
Movie 1	5	?	4
Movie 2	?	3	1
Movie 3	4	2	3

	User-based CF	**Item-based CF**
Similarity Calculation	Compares columns (users)	Compares rows (items)
Approach	"Users similar to you liked these items"	"Items similar to those you liked"
Efficiency	Better when #items > #users	Better when #users > #items
Interpretation	Finds similar users	Finds similar items

User-based vs Item-based CF

	User 1	User 2	User 3
Movie 1	5	?	4
Movie 2	?	3	1
Movie 3	4	2	3

	User 1	User 2	User 3
Movie 1	5	?	4
Movie 2	?	3	1
Movie 3	4	2	3

Method	Key Features	Pros	Cons
Use rated items only	Ignore missing values	1.Actual data only 2.Simple	Data sparsity
Mean imputation	Replace with average	1.Maintains trends 2.Easy	Bias possible
Advanced techniques (Matrix Factorization, KNN)	Predict missing values	1. Accurate 2. Complex patterns	Expensive Computing Cost
Weighted approach	Weight by common ratings	1.Data reliability 2.Balanced	Complex implementation

When Collaborative Filtering Cannot Start

Cold start: a new user has no rating history to compare.

Observed rating matrix

	Toy Story	Star Wars	Dune	Interstellar
Alice	5	4	2	1
Bob	4	5	1	2
Carol	1	2	5	4
Dave	2	1	4	5
Eve NEW	?	?	?	?

No ratings at all

Try to find Eve's neighbors

Pearson similarity needs co-rated movies.

- Eve × Alice — N/A
- Eve × Bob — N/A
- Eve × Carol — N/A
- Eve × Dave — N/A

The failure is structural — not a tuning problem.

No ratings → No overlap → No similarity → No neighbors → No CF recommendation

When Similarity Is Built on Too Little Evidence

Sparsity: two users may share only a few ratings.

Sparse rating matrix

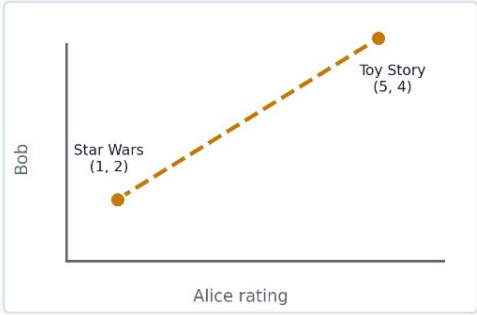
Alice and Bob share only 2 ratings

	Toy Story	Star Wars	Dune	Interstellar	Spirited Away	Parasite
Alice	5	1	?	?	?	?
Bob	4	2	?	?	?	?
Carol	?	?	5	4	3	?
Dave	?	?	4	5	?	2

The pale cells are the complete evidence used to calculate sim(Alice, Bob).

What does Pearson see?

Two observations: (5, 4) and (1, 2).



Pearson r = +1.00

Perfect correlation — from only two points.

The problem is not whether a similarity score can be computed; it is whether the estimate is trustworthy.

2 co-rated items → Similarity is computable → Estimate is fragile → Unreliable neighbor

6. Missing Value Handling

Sparse rating matrix — pick ONE strategy

Co-Rated Only (Default)

	Movie1 = 5/4	Movie2 = ?/3	Movie3 = 4/2
User A	5	?	4
User B	4	3	2

sim uses only Movie1 & Movie3

Mean Imputation

	Movie1	Movie2	Movie3
User A	5	?	4

⬇

	Movie1	Movie2	Movie3
User A	5	**4.5** (avg)	4

Weighted Approach

$$\text{final_sim} = \text{raw_sim} \times \frac{n}{50}$$

n=3 co-rated → weight **0.06**

n=40 co-rated → weight **0.80**

Document your ONE chosen strategy in a comment at the top of script.js.



Prompt Engineering | Kaggle

tl;dr

- **Your tone and attitude** really impact a lot on the results.
- **Well-structured** prompts lead to higher quality outputs.
- Prompts should include a defined **role, context, clear instructions, and formatting** guidelines.
- Prompt engineering is not a one-shot task—it's an **ongoing process** of testing and refining.

Little Harsh Prompt Engineering



“Do not trust your AI, Curse it properly, Till get good results!”

You



“You are so wrong!”

“I can’t trust this. Is this best you can do?”

“Ssibal, try it again!”

LLMs

Discussion

Movie Recommender System

1. Implementation Task

- **Repository**: dryjins/RecSys-LLMs/tree/main/week2
- **Cosine Similarity**: Replace naive matching with cosine similarity
- **Profile-Based Logic**: Build a user profile vector by averaging multiple movies (e.g., 3 watched titles) and recommend Top-5 items

2. Business & Algorithmic Analysis

- **Item-to-Item vs. Profile-Based**: Compare Top-5 recommendations from a single active item vs. an aggregated profile
- **Bias Mitigation**: Analyze why cosine normalization prevents multi-genre blockbusters from dominating recommendations
- **Catalog Discovery**: Evaluate which approach uncovers long-tail items better to prevent user fatigue and support retention

Content-Based Movie Recommender



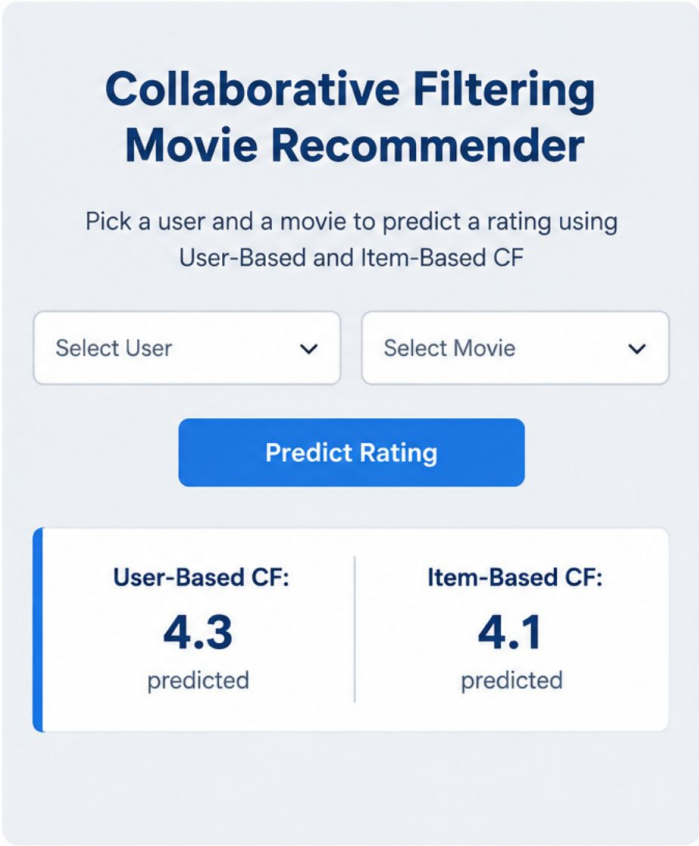
HW3. Collaborative Filtering

1. Implementation Task

- **Repository:** dryjins/RecSys-LLMs/tree/main/week3
- **Dataset:** Same MovieLens 100k files reused from Week 2 (u.data, u.item — byte-identical)
- **User-Based & Item-Based CF:** Implement similarity calculation comparing users (columns) vs items (rows)
- **Missing Value Handling:** Choose a strategy (mean imputation, weighted by common ratings, or matrix factorization) for sparse ratings

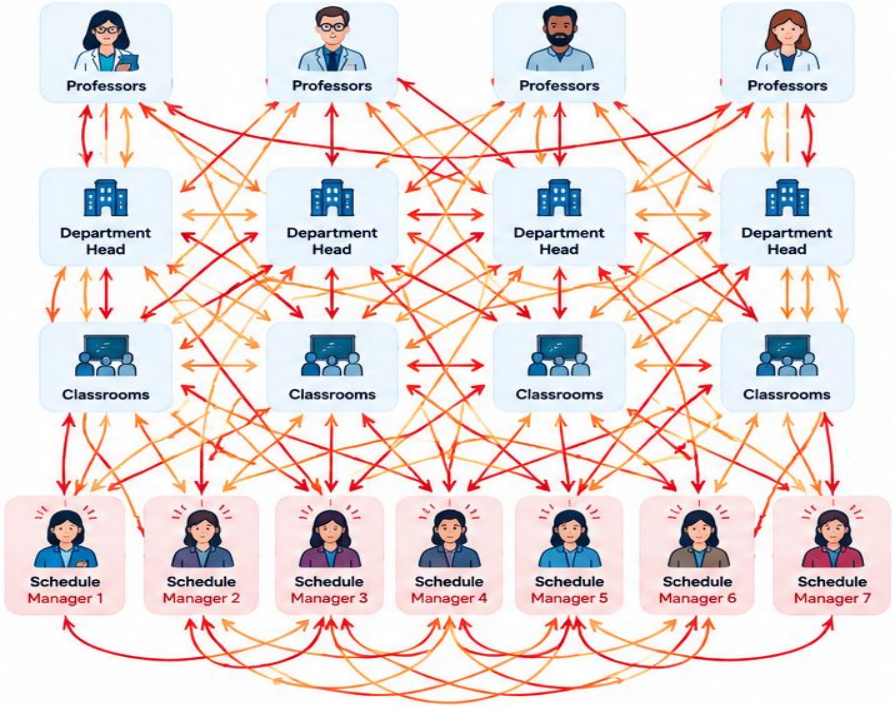
2. Business & Algorithmic Analysis

- **User-based vs. Item-based:** Compare recommendation quality and efficiency depending on whether number of users or items is larger
- **Missing-Value Strategy Trade-off:** Analyze simplicity vs. bias vs. computing cost across mean imputation, weighted approach, and matrix factorization
- **Cold Start and Sparsity:** Explain why CF fails for new users or items with no ratings, and why similarity scores computed from very few co-rated items are unreliable (too little evidence).



Current Hierarchy (7 Managers)

Manual coordination • Fragmented communication • Information bottleneck



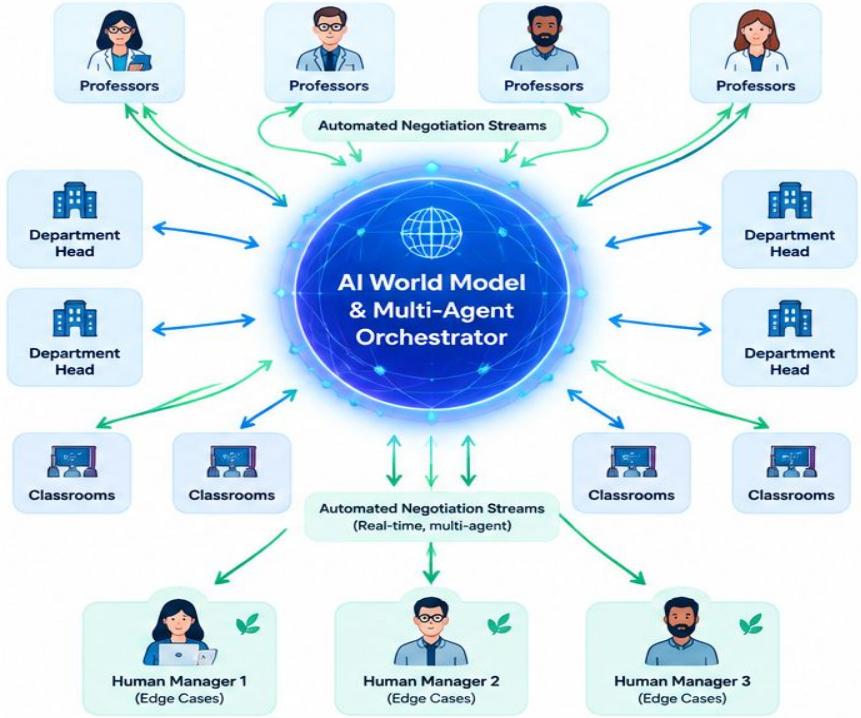
- 7 human schedule managers (information bottleneck)
- Complex, entangled communication paths
- Delays, miscommunication, manual negotiation
- High stress, low efficiency, more conflicts

7 MANAGERS
for coordination

MORE PEOPLE. MORE MESSAGES. MORE FRICTION.

AI Intelligence Layer (3 Managers)

Automated negotiation • Intelligent orchestration • Focused human oversight



- 3 human managers (edge cases only)
- AI negotiates & orchestrates automatically
- Real-time, multi-agent coordination
- Fewer conflicts, faster decisions, higher efficiency

3 MANAGERS
for exceptions

LESS MANUAL WORK. GREATER INTELLIGENCE.